

## **Лекция 4. Двумерная графика. Создание игровых приложений.**

- Потоки, таймеры.
- Управление звуком.
- Графические примитивы.
- Обработка нажатий TouchPad.
- Лямбда выражения в Kotlin.

### **Потоки и таймеры: управление временем и параллелизмом в играх**

В разработке 2D-игр необходимо управлять временем и параллелизмом, чтобы обеспечить плавную анимацию, обработку игровой логики и реакцию на действия пользователя. Для этого в Android используются потоки (Thread) и таймеры (Handler, Timer) .

Потоки позволяют выполнять длительные операции (например, обновление состояния игры) вне UI-потока, который отвечает за отрисовку интерфейса. Это предотвращает «подвисание» экрана при выполнении ресурсоёмких задач. Таймеры обеспечивают периодическое выполнение действий, таких как обновление позиций объектов, проверка условий победы или смена уровней.

Пример 1: Анимация движения круга с помощью потока. Этот пример демонстрирует реализацию анимации движения круга на экране через Thread.

```
class GameView(context: Context) : View(context) {  
    private var x = 0f  
    private var y = 0f  
    private val paint = Paint().apply {  
        color = Color.RED  
        style = Paint.Style.FILL  
    }  
    private val gameThread = Thread {  
        while (true) {  
            // Обновление координат круга  
            x += 5f
```

```

        y += 5f
        // Перерисовка экрана
        postInvalidate()
        Thread.sleep(16) // ~60 FPS
    }
}

override fun onDraw(canvas: Canvas) {
    super.onDraw(canvas)
    canvas.drawCircle(x, y, 50f, paint)
}

fun startGame() {
    gameThread.start()
}
}

```

- `GameView` — кастомный класс, наследующий `View`, где реализуется логика отрисовки и анимации.
- `gameThread` — фоновый поток, обновляющий координаты круга и вызывающий перерисовку через `postInvalidate()`.
- `Thread.sleep(16)` задаёт интервал между кадрами (~60 FPS), обеспечивая плавное движение.
- `postInvalidate()` безопасно вызывает перерисовку из фонового потока, обновляя экран.

Использование потока требует аккуратности, чтобы избежать блокировки UI-потока. То есть метод `postInvalidate()` гарантирует, что перерисовка произойдёт в UI-поток, предотвращая ошибки синхронизации. Такой подход подходит для простых игр, где логика и анимация не слишком сложны.

Пример 2: Таймер для обновления состояния игры. Этот пример использует `Handler` для периодической проверки игровых событий, таких как обновление счёта или изменение уровня.

```

class MainActivity : AppCompatActivity() {
    private lateinit var handler: Handler
    private var score = 0

    override fun onCreate(savedInstanceState: Bundle?) {

```

```

super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)

handler = Handler(Looper.getMainLooper())
val updateRunnable = object : Runnable {
    override fun run() {
        score++
        findViewById<TextView>(R.id.scoreText).text = "Очки: $score"
        handler.postDelayed(this, 1000) // Обновление раз в секунду
    }
}
handler.post(updateRunnable)
}
override fun onDestroy() {
    super.onDestroy()
    handler.removeCallbacksAndMessages(null) // Очистка таймера
}
}

```

- Handler с Looper.getMainLooper() работает в UI-поток, обновляя счёт каждую секунду.
- postDelayed() реализует цикл, аналогичный таймеру, обновляя счёт и вызывая повторный запуск.
- onDestroy() очищает таймер, предотвращая утечки памяти при закрытии активности.

Handler удобен для задач, требующих регулярных обновлений, например, подсчёта времени или проверки условий. В сложных играх может потребоваться комбинировать потоки и таймеры для оптимального распределения нагрузки.

Регулярные обновления должны быть сбалансированы с частотой отрисовки, чтобы избежать дисбаланса между логикой и визуализацией.

### **Управление звуком: интеграция аудио в игровую среду**

Звуковые эффекты и фоновая музыка усиливают игровой опыт, создавая эмоциональную вовлечённость и обратную связь для пользователя. В Android для воспроизведения аудио используются MediaPlayer (для длительного воспроизведения) и SoundPool (для коротких звуковых эффектов).

Пример 1: Воспроизведение фоновой музыки через MediaPlayer.

```

class GameActivity : AppCompatActivity() {
    private lateinit var mediaPlayer: MediaPlayer

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_game)

        mediaPlayer = MediaPlayer.create(this, R.raw.background_music)
        mediaPlayer.isLooping = true // Бесконечное воспроизведение
        mediaPlayer.start()
    }
    override fun onDestroy() {
        super.onDestroy()
        mediaPlayer.release() // Освобождение ресурсов
    }
}

```

- MediaPlayer используется для фоновой музыки, так как он поддерживает длительное воспроизведение и циклическое повторение.
- isLooping = true позволяет музыке играть бесконечно, создавая постоянную атмосферу.
- release() освобождает ресурсы после окончания игры, предотвращая утечки памяти.

MediaPlayer требует точного управления ресурсами, особенно при смене активностей или приостановке игры. Фоновая музыка должна быть с низким потреблением ресурсов, чтобы не мешать основной логике.

Пример 2: Воспроизведение звукового эффекта через SoundPool

```

class GameView(context: Context) : View(context) {
    private lateinit var soundPool: SoundPool
    private var soundId = 0

    init {
        soundPool = SoundPool.Builder().build()
        soundId = soundPool.load(context, R.raw.jump_sound, 1)
    }
    override fun onDraw(canvas: Canvas) {
        // Логика игры...
        // Вызов звука при действии
        soundPool.play(soundId, 1f, 1f, 0, 0, 1f)
    }
}

```

```

    }
    fun releaseSounds() {
        soundPool.release()
    }
}

```

SoundPool оптимизирован для коротких звуковых эффектов, таких как прыжки или выстрелы. Здесь load() загружает звук из ресурсов, готовя его к немедленному воспроизведению, а play() запускает звук с указанием громкости и приоритета.

SoundPool эффективен для множества одновременных звуков, но ограничен по длительности воспроизведения. Воспроизведение звуков должно быть синхронизировано с игровыми событиями, чтобы избежать дублирования или пропусков.

### **Графические примитивы: основа визуализации в 2D-играх**

Работа с графикой в Android осуществляется через класс Canvas, который позволяет рисовать фигуры, текст, изображения. Это основа для создания игрового мира, персонажей и интерфейсов.

Пример 1: Отрисовка прямоугольника и текста

```

class GameCanvasView(context: Context) : View(context) {
    private val paint = Paint()

    override fun onDraw(canvas: Canvas) {
        super.onDraw(canvas)
        // Рисование прямоугольника
        paint.color = Color.BLUE
        canvas.drawRect(100f, 100f, 300f, 200f, paint)

        // Рисование текста
        paint.color = Color.BLACK
        paint.textSize = 40f
        canvas.drawText("Игра началась!", 120f, 300f, paint)
    }
}

```

Здесь Canvas.drawRect() рисует прямоугольник, например, платформу для персонажа, Canvas.drawText() выводит текст, такой как счёт или сообщения, а Paint настраивает цвет, размер текста и стиль отрисовки. Canvas работает на низком уровне, требуя точного расчёта координат и параметров. Для

сложных графических сцен может потребоваться оптимизация, например, кэширование изображений.

Пример 2: Анимация перемещения круга с использованием Handler

```
class AnimatedCircleView(context: Context) : View(context) {
    private var x = 100f
    private val radius = 50f
    private val paint = Paint().apply {
        color = Color.GREEN
    }

    private val handler = Handler(Looper.getMainLooper())
    private val updateRunnable = object : Runnable {
        override fun run() {
            x += 10f
            if (x > width) x = 0f
            invalidate() // Перерисовка
            handler.postDelayed(this, 16) // ~60 FPS
        }
    }

    override fun onSizeChanged(w: Int, h: Int, oldw: Int, oldh: Int) {
        super.onSizeChanged(w, h, oldw, oldh)
        handler.post(updateRunnable)
    }

    override fun onDraw(canvas: Canvas) {
        super.onDraw(canvas)
        canvas.drawCircle(x, height / 2f, radius, paint)
    }

    override fun onDetachedFromWindow() {
        super.onDetachedFromWindow()
        handler.removeCallbacks(updateRunnable) // Остановка анимации
    }
}
```

Handler и Runnable реализуют анимацию круга, обновляя его координаты, метод `invalidate()` вызывает перерисовку, обновляя позицию круга, а `onSizeChanged()` гарантирует, что анимация начнётся только после определения размеров экрана.

Анимация через Handler подходит для простых сценариев, но в сложных играх может быть заменена на `SurfaceView`. Частота обновления (16 мс) соответствует 60 кадрам в секунду, обеспечивая плавность.

## Обработка нажатий TouchPad: взаимодействие с пользователем

В играх важно обрабатывать касания экрана для управления персонажем, запуска действий или взаимодействия с объектами.

Пример 1: Управление персонажем через касание

```
class GameSurface(context: Context) : View(context) {
    private var playerX = 100f
    private var playerY = 100f
    private val playerPaint = Paint().apply {
        color = Color.YELLOW
    }

    override fun onTouchEvent(event: MotionEvent): Boolean {
        when (event.action) {
            MotionEvent.ACTION_DOWN -> {
                playerX = event.x
                playerY = event.y
                invalidate()
            }
        }
        return true
    }
    override fun onDraw(canvas: Canvas) {
        super.onDraw(canvas)
        canvas.drawCircle(playerX, playerY, 30f, playerPaint)
    }
}
```

Здесь метод `onTouchEvent()` обрабатывает касания, обновляя позицию персонажа, `event.x` и `event.y` передают координаты касания, которые можно использовать для перемещения или взаимодействия, а метод `invalidate()` запускает перерисовку, обновляя позицию персонажа. Необходимо учесть, что обработка касаний должна быть чувствительной к типу событий (`ACTION_DOWN`, `ACTION_MOVE`, `ACTION_UP`). В сложных играх может потребоваться реализация жестов или виртуальных джойстиков.

Пример 2: Обнаружение жестов (движение пальца)

```
class GestureView(context: Context) : View(context) {
    private var path = Path()
    private val paint = Paint().apply {
        color = Color.RED
        strokeWidth = 5f
    }
```

```

        style = Paint.Style.STROKE
    }

    override fun onTouchEvent(event: MotionEvent): Boolean {
        when (event.action) {
            MotionEvent.ACTION_DOWN -> path.moveTo(event.x, event.y)
            MotionEvent.ACTION_MOVE -> path.lineTo(event.x, event.y)
            MotionEvent.ACTION_UP -> {
                path.reset() // Сброс для нового жеста
            }
        }
        invalidate()
        return true
    }

    override fun onDraw(canvas: Canvas) {
        super.onDraw(canvas)
        canvas.drawPath(path, paint)
    }
}

```

В этом примере Path хранит траекторию движения пальца, которая отрисовывается на экране, ACTION\_DOWN , ACTION\_MOVE , ACTION\_UP обрабатывают начало, движение и завершение жеста, а метод drawPath() отрисовывает траекторию, например, для рисования или жестового управления.

Жесты могут быть использованы для сложных действий, таких как броски, свайпы или рисование. Точность обработки касаний критична для игрового процесса, требуя тщательной настройки.

### **Лямбда-выражения в Kotlin: упрощение игровой логики**

Лямбда-выражения в Kotlin позволяют писать краткий и выразительный код, особенно полезный для обработки событий, анимаций и игровой логики. Они упрощают интеграцию с API Android, уменьшая boilerplate-код.

Пример 1: Обработка клика с лямбдой

```

val button = Button(this).apply {
    text = "Прыгнуть"
    setOnClickListener {
        // Логика прыжка персонажа
        playerY -= 50f
    }
}

```



```

        invalidate()
    }
}

```

В данном примере `setOnClickListener { ... }` использует лямбду для обработки клика, заменяя анонимный класс. При нажатии кнопки обновляется позиция персонажа (`playerY`) и вызывается перерисовка.

Таким образом, лямбды упрощают работу с событиями, делая код чище и понятнее. Они особенно эффективны для коротких действий, таких как обработка нажатий или запуск анимаций.

#### Пример 2: Анимация с лямбдой

```

val animator = ObjectAnimator.ofFloat(view, "translationX", 0f, 100f)
animator.duration = 500
animator.addUpdateListener { animation ->
    val value = animation.animatedValue as Float
    // Логика обновления позиции объекта
}
animator.start()

```

Здесь `ObjectAnimator` реализует анимацию перемещения объекта, а метод `addUpdateListener()` использует лямбду для обработки каждого шага анимации, `animatedValue` — возвращает текущее значение анимации, которое можно применить к игровым объектам. Лямбды упрощают работу с анимациями, обработкой событий и логикой игры, они интегрируются с Android SDK, улучшая читаемость и упрощая поддержку кода.